

Unrolled List

–Display List

–Space Complexity Analysis and Algorithm

Program:

```
void displayList() {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    cout << "\n--- Unrolled Linked List Content ---\n";
    while (temp != NULL) {
        cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
        for (int i = 0; i < temp->numElements; i++) {
            cout << temp->elements[i];
            if (i < temp->numElements - 1) {
                cout << ", ";
            }
        }
        cout << "]\n";

        temp = temp->next;
        nodeCount++;
    }
    cout << "-----\n";
}

.....

    case 3:
        displayList();
        break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the total memory occupied by the data that the algorithm needs to process. We look at this from two perspectives:

A. Parametric Input Space: $O(1)$

This measures the memory required exclusively for the arguments passed directly into the function signature.

- *The function signature is `void displayList()`.*
- *Unlike `insertElement(int val)` or `deleteElement(int val)`, this function accepts zero parameters.*
- *Because no parameters are passed into the function, it requires absolutely no memory for arguments.*
- **Result:** *The parametric input space is strictly $O(1)$ (Constant Space).*

B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm operates on (the unrolled linked list).

- *The function traverses a globally defined data structure accessed via the `head` pointer.*
- *The total memory occupied by this entire list scales linearly with the number of existing nodes.*

- *For a list containing n elements, the data is grouped into*

$$N \approx \left\lceil \frac{n}{4} \right\rceil \text{ nodes.}$$

- *Because the total memory footprint physically grows as more elements and nodes are added to the list, the memory requirement scales linearly.*
- *Result: The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$ i. e. $[O(N) \equiv O(n)]$).*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space measures the temporary, extra memory that the algorithm actively creates to perform its task, entirely separate from the input data itself.

*Let's look at the exact variables declared inside the **displayList** function to see how much extra memory it asks the computer for:*

- *One temporary pointer: **struct Node * temp = head;** used to traverse the list.*
- *One counter variable: **int nodeCount = 1;** used to track the current node number.*
- *One loop iterator: **int i = 0;** used for the inner **for** loop to print elements.*

Key Observations:

1. **No Dynamic Allocation:** *The function only reads data. It never uses `malloc()` or `new` to create additional nodes or arrays.*
2. **No Recursion:** *The function relies on simple iterative loops (while and for), meaning the Call Stack does not grow.*

Result: *Because the extra memory required by the function is fixed and does not change based on the size of the list, the Auxiliary Space Complexity is strictly **$O(1)$ (Constant Space)**.*

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The *for* loop is governed by the condition $i < N$,
meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

Inside the loop, the *malloc* function is called during every single iteration.

malloc physically claims a brand – new
block of memory from the computer's heap.

3. Total Memory Consumed

If *sizeof(struct Node)* takes a fixed constant amount of memory
(for example, 24 bytes), the total extra memory generated
by this loop is:

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

In Big – O notation, we drop the exact hardware constants
(like 24 bytes) because we only care about how the memory
requirements grow as N gets larger. Because the memory requested
scales linearly with the variable N , the complexity is $O(N)$.

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

= ($O(1) + O(N)$) + $O(1)$

= $O(N)$.

or,

($O(1) + O(n)$) + $O(1)$

= $O(n)$.

Algorithm: UnrolledDISPList()

1. [CHECK IF LIST IS EMPTY]

- **IF START = NULL, Then:**
 - **Write: "The list is empty."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL VARIABLES]

- *SET TEMP_PTR := START*
- *SET nodeCount := 1*
- *Write: " — — — Unrolled Linked List Content — — — "*

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE *TEMPPTR* $\neq$ NULL:**

4. *[PRINT NODE HEADER]*

- **Write:** *Node*, *nodeCount*, (*numElements*[*TEMPPTR*],/,*MAX_ELEMENTS*, *elements*): [

5. [INNER LOOP: ELEMENT DISPLAY]

- *SET* $i := 0$
- *REPEAT WHILE* $i < \text{numElements}[\text{TEMPPTR}]$:
 - *Write*: $\text{elements}[\text{TEMPPTR}][i]$

(Check if a comma is needed)

- *IF* $i < \text{numElements}[\text{TEMPPTR}] - 1$, *Then:*
 - *Write:* ", "
 - [*End of inner If structure.*]
 - *SET* $i := i + 1$
- [*End of Inner Display Loop*]

6. [FINALIZE NODE AND MOVE TO NEXT]

- ***Write:***] (*Closes the bracket for the current node and starts a new line*)
 - ***SET TEMPPTR := LINK[TEMPPTR]*** (*Move to the next node block*)
 - ***SET nodeCount := nodeCount + 1*** (*Increment the node tracker*)
 - [***End of Outer Node Traversal Loop***]
- Write:*** " _ _ _ _ _ "

7. [END OF ALGORITHM]